What is Gradient Boosting?

Authors

Bryan Clark
Senior Technology Advocate

Fangfang Lee
Developer Advocate
IBM

What is gradient boosting?

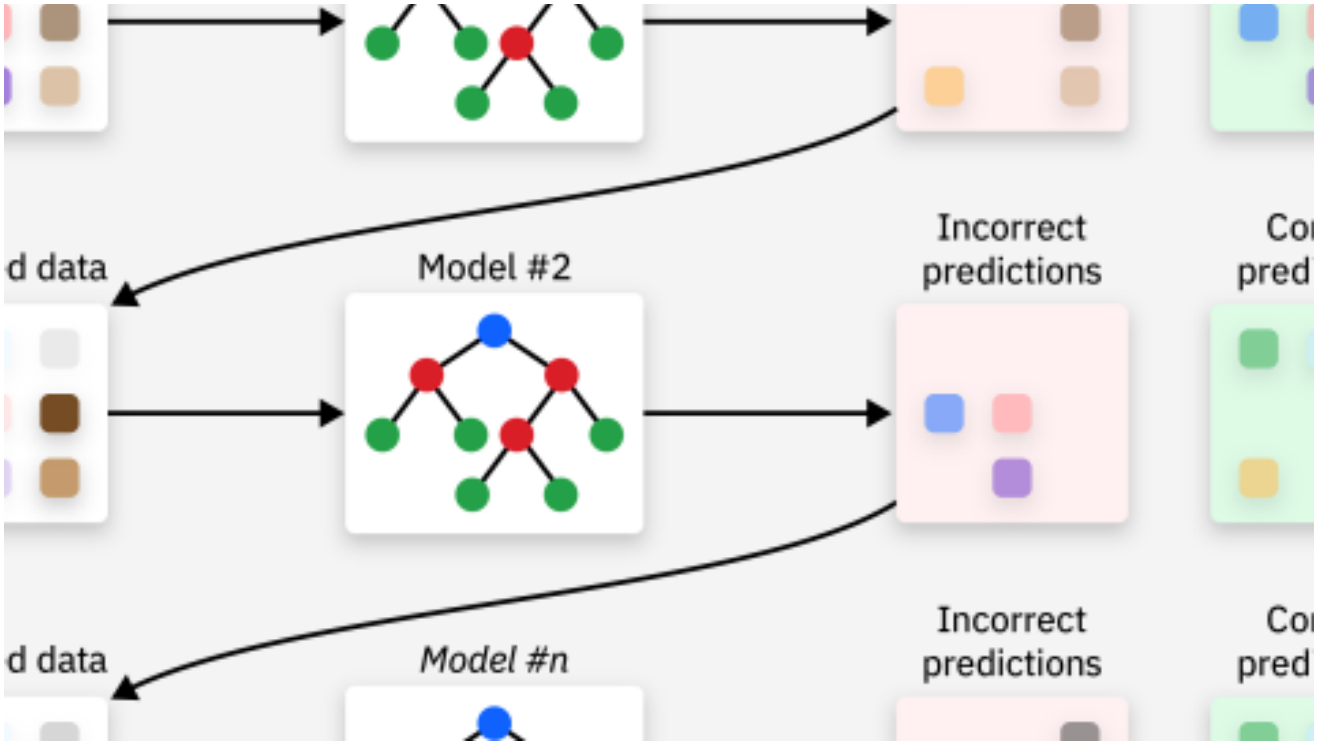
Gradient boosting is an ensemble learning algorithm that produces accurate predictions by combining multiple decision trees into a single model. This algorithmic approach to predictive modeling, introduced by Jerome Friedman, uses base models to build upon their strengths, correcting errors and improving predictive capabilities. By capturing complex patterns in data, gradient boosting excels at diverse predictive modeling tasks.
[1]

Ensemble learning and boosting

Ensemble learning is a machine learning approach that combines multiple models or methods to boost predictive performance. It often employs techniques such as bagging and boosting. Bagging involves training numerous models on different data subsets with some randomness, which helps reduce variance by averaging out individual errors. A great example of this approach is random forests.

In contrast, boosting is an ensemble technique that iteratively trains models to correct previous mistakes. It gives more weight to misclassified instances in subsequent models, allowing them to focus on challenging data points and ultimately enhancing overall performance. AdaBoost, widely regarded as the first applicable boosting algorithm, is a classic illustration of this method. Both bagging and boosting optimize the bias variance tradeoff in models, leading to more robust performance. [2]

These techniques are extensively used in machine learning to improve model accuracy, especially when dealing with complex or noisy datasets. By combining multiple perspectives, ensemble learning provides a way to overcome the limitations of individual models and achieve improved optimization. [3]



How gradient boosting works

Gradient boosting is a machine learning technique that combines multiple weak prediction models into a single ensemble. These weak models are typically decision trees, which are trained sequentially to minimize errors and improve accuracy. By combining multiple decision tree regressors or decision tree classifiers, gradient boosting can effectively capture complex relationships between features.

One of the key benefits of gradient boosting is its ability to iteratively minimize the loss function, resulting in improved predictive accuracy. However, one must be conscious of overfitting, which occurs when a model becomes too specialized to the training data and fails to generalize well to new instances. To mitigate this risk, practitioners must carefully tune hyperparameters, monitor model performance during training and employ techniques like regularization, pruning or early stopping. By understanding these challenges and taking steps to address them, practitioners can successfully harness the power of gradient boosting—including the use of regression trees—to develop accurate and robust prediction models for various applications. [4,5]

Mean Squared Error (MSE) is one loss function used to evaluate how well a machine learning model's predictions match actual data. MSE calculates the average of the squared differences between the predicted and observed values. The formula for MSE is: $MSE = \Sigma(yi - pi)^2 / n$, where yi represents the actual value, pi is the predicted value, and n is the number of observations.

Expanding a bit further, MSE quantifies the difference between predicted values and actual values represented in the dataset for regression problems. The squaring step helps ensure that both positive and negative errors contribute to the final value without canceling each other out. This method gives more weight to larger errors, as the errors are squared.

To interpret MSE, generally a lower value indicates better agreement between predictions and observations. However, achieving a lower MSE is difficult in real-world scenarios due to the inherent randomness that exists not just in the dataset but in the population. Instead, comparing MSE values over time or across different models can help determine improvements in predictive accuracy. It is also important to note that specifically aiming for an MSE of zero is almost always indicative of overfitting. [6]

Some popular implementations of boosting methods within Python include Extreme Gradient Boosting (XGBoost) and Light Gradient-Boosting Machine (LightGBM). XGBoost is designed for speed and performance and is used for regression and classification problems. LightGBM used tree-based learning algorithms and is suited for large-scale data processing. Both methods further enhance accuracy, especially when grappling with intricate or noisy datasets. LightGBM employs a technique called Gradient-based One-Side Sampling (GOSS) to filter out the data instances for finding the split points, significantly reducing computational overhead. Integrating multiple ensemble learning techniques, remove the constraints of individual models and attain superior results in data science scenarios. [7,8]

The following is a step-by-step breakdown of how the gradient boosting process works.

Initialization: Starts by using a training set to establish a foundation with a base learner model, often a decision tree, whose initial predictions are randomly generated. Typically, the decision tree will only contain a handful of leaf nodes or terminal nodes. Often chosen due to their interpretability, these weak or base learners serve as an optimal starting point. This initial setup paves the way for subsequent iterations to build upon.

Calculating residuals: For each training example, calculate the residual error by subtracting the predicted value from the actual value. This step identifies areas where the model's predictions need improvement.

Refining with regularization: Post residual calculation and preceding the training of a new model, the process of regularization takes place. This stage involves downscaling the influence of each new weak learner integrated into the ensemble. By carefully calibrating this scale, one can govern how swiftly the boosting algorithm advances, thereby aiding in overfitting prevention and overall performance optimization.

Training the next model: Use the residual errors calculated in the previous step as targets and train a new model or weak learner to predict them accurately. This step's focus is on correcting the mistakes made by the previous models, refining the overall prediction.

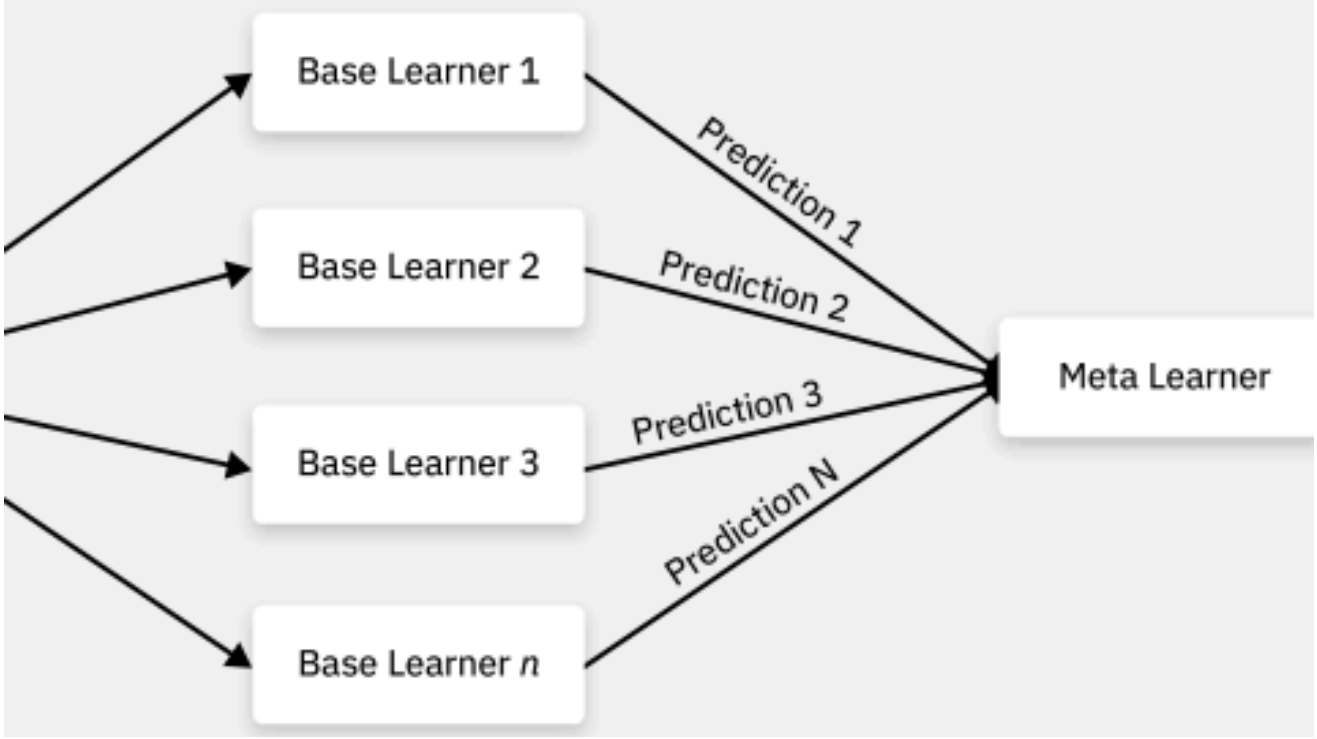
Ensemble updates: In this stage, the performance of the updated ensemble (including the newly trained model) is typically evaluated by using a separate test set. If the performance on this holdout dataset is satisfactory, the ensemble can be updated by incorporating the new weak learner; otherwise, adjustments might be necessary to the hyperparameters.

Repetition: Repeat the previously presented steps as necessary. Each iteration builds upon and refines the base model through the training of new trees, further improving the model's accuracy. If the ensemble update and final model is satisfactory compared to the baseline model based on accuracy, then move to the next step.

Stopping criteria: Stop the boosting process when a predetermined stopping criterion is met, such as a maximum number of iterations, target accuracy or diminishing returns. This step helps ensure that the model's final prediction achieves the expected balance between complexity and performance.

Emsemble methods and stacking

Combining gradient boosting with other machine learning algorithms through ensemble methods or stacking can further improve predictive accuracy. For example, blending gradient boosting with support vector machines (SVMs), random forests, or k-nearest neighbors (KNN) can leverage the strengths of each model and create a more robust ensemble. Stacking involves training multiple base learners and by using their outputs as inputs for a meta learner, which combines predictions to generate final outputs. [9]



Early stopping and cross-validation

Monitoring model performance during training and implementing early stopping techniques can help prevent overfitting by halting the boosting process once performance on a validation set stops improving or begins degrading. Additionally, using cross-validation strategies such as k-fold cross-validation can provide more reliable estimates of model performance and hyperparameter tuning, further enhancing gradient boosting's predictive capabilities.

Addressing imbalanced datasets

Gradient boosting is sensitive to class imbalance, which can lead to biased predictions favoring the majority class. To address this issue, practitioners can employ techniques such as oversampling the minority class, undersampling the majority class or by using weighted loss functions that assign higher penalties for misclassifying minority instances.

By implementing these strategies and carefully tuning hyperparameters, practitioners can significantly enhance gradient boosting's predictive accuracy and robustness across various applications, from high-dimensional data analysis to complex environmental monitoring tasks.

Gradient boosting hyperparameter tuning in scikit-learn (sklearn)

The GradientBoostingClassifier and GradientBoostingRegressor in scikit-learn offer a versatile approach to implementing the gradient boosting algorithm, catering to both classification and regression tasks. By allowing users to fine-tune several parameters, these implementations enable customization of the boosting process according to specific requirements and data characteristics.

Tree depth (max_depth): Controls the maximum depth of individual decision trees and should be tuned for best performance. Deeper trees can capture more complex relationships but are also prone to overfitting.

Learning rate (learning_rate): Determines the contribution of each tree to the overall ensemble. A smaller learning rate slows down convergence and reduces the risk of overfitting, while a larger value might lead to faster training at the expense of potential overfitting.

Number of trees (n_estimators): Specifies the total number of trees in the ensemble. Increasing this parameter can improve performance but also increases the risk of overfitting.

Additionally, scikit-learn's gradient boosting implementations provide out-of-bag (OOB) estimates, a technique for assessing model performance without requiring separate validation datasets. Furthermore, staged prediction methods in scikit-learn enable incremental predictions as new data becomes available, making real-time processing possible and efficient. In summary, scikit-learn's gradient boosting implementations provide a rich set of features for fine-tuning models according to specific needs and dataset characteristics, ultimately fostering superior predictive performance. [10]

Gradient boosting use cases

Handling high-dimensional medical data: Gradient boosting is capable of effectively dealing with datasets containing many features relative to the number of observations. For instance, in medical diagnosis, gradient boosting can be used to diagnose diseases based on patient data, which might contain over 100 features. By leveraging decision trees as weak learners, the algorithm might be able to manage high dimensionality, where traditional linear regression models might struggle. The algorithm might also extract valuable information from sparse data, making it suitable for applications such as bioinformatics or text classification problems. [11,12]

Reduce customer service churn rates: When a model already exists but performance is suboptimal, gradient boosting can be employed to iteratively refine predictions by correcting previous errors. One example is predicting customer churn in telecommunications, where a traditional logistic regression model was used. The company can apply gradient boosting algorithms to identify key factors contributing to customers leaving for another service, such as high call volumes or poor network performance. By incorporating these factors into the model, they might be able to improve accuracy and reduce churn rates. [13]

Predicting beech tree survival: In a forest ecosystem, beech leaf disease (BLD) is a significant threat to beech tree health. Researchers might develop a predictive model to identify trees at risk of BLD and predict their likelihood of survival. A machine learning model might be developed that can analyze environmental factors such as climate data, soil quality and tree characteristics to compute the likelihood of beech tree survival (BTS) over a 5-year period. By using gradient boosting techniques, it is possible to capture intricate patterns that might be overlooked by simpler methods. The model might identify trees at risk of BLD with high precision and forecast their BTS accurately, empowering researchers to prioritize interventions and protect vulnerable beech trees effectively. This use case demonstrates how gradient boosting can enhance the predictive power of machine learning models in complex environmental monitoring tasks. [14]

Ebook

Unlock the power of generative AI + ML

Learn how to confidently incorporate generative AI and machine learning into your business.

Read the ebook →

Resources

Report

IBM is named a Leader in Data Science & Machine Learning

Learn why IBM has been recognized as a Leader in the 2025 Gartner® Magic Quadrant™ for Data Science and Machine Learning Platforms.

AI models

Explore IBM Granite

IBM Granite is our family of open, performant and trusted AI models, tailored for business and optimized to scale your AI applications. Explore language, code, time series and guardrail options.

Report

AI in Action 2024

We surveyed 2,000 organizations about their AI initiatives to discover what's working, what's not and how you can get ahead.

Explainer

Supervised learning models

Explore supervised learning approaches such as support vector machines and probabilistic classifiers.

1 / 2

Related solutions

IBM watsonx.ai
Train, validate, tune and deploy generative AI, foundation models and machine learning capabilities with IBM watsonx.ai, a next-generation enterprise studio for AI builders. Build AI applications in a fraction of the time with a fraction of the data.

Explore watsonx.ai →

Artificial intelligence solutions
Put AI to work in your business with IBM's industry-leading AI expertise and portfolio of solutions at your side.

Explore AI solutions →

AI consulting and services
Reinvent critical workflows and operations by adding AI to maximize experiences, real-time decision-making and business value.

Explore AI services →

Take the next step

Get one-stop access to capabilities that span the AI development lifecycle. Produce powerful AI solutions with user-friendly interfaces, workflows and access to industry-standard APIs and SDKs.

Explore watsonx.ai →

Book a live demo →

Footnotes

[1] Friedman, Jerome H. “Greedy Function Approximation: A Gradient Boosting Machine.” The Annals of Statistics 29, no. 5 (2001): 1189–1232. http://www.jstor.org/stable/2699986.

[2] Schapire, R.E. (2013). Explaining AdaBoost. In: Schölkopf, B., Luo, Z., Vovk, V. (eds) Empirical Inference. Springer, Berlin, Heidelberg. https://link.springer.com/chapter/10.1007/978-3-642-41136-6_5

[3] Fan, Wenjie, et al. "A Survey of Ensemble Learning: Recent Trends and Future Directions." arXiv preprint arXiv:2501.04871 (2025).

[4] Matsubara, Takuo. “Wasserstein Gradient Boosting: A Framework for Distribution- Valued Supervised Learning.” arXiv.org, August 29, 2024. https://search.arxiv.org/paper.jsp?r=2405.09536&qid=1743170618344ler_nCn N_-2014411830&qs=gradient%2Bboosting.

[5] Emami, Seyedsaman, and Gonzalo Martínez-Muñoz. 2023. “Sequential Training of Neural Networks with Gradient Boosting.” IEEE Access 11 (January): 42738–50. https://ieeexplore.ieee.org/document/10110967

[6] Chen, Tianqi, et al. "Mean Squared Error." Encyclopedia Britannica, 2023. https://www.britannica.com/science/mean-squared-error.

[7] XGBoost Developers. "XGBoost: A Scalable Tree Boosting System." GitHub, 2021. https://github.com/dmlc/xgboost/blob/master/README.md .

[8] LightGBM Documentation Team. "LightGBM." 2021. https://lightgbm.readthedocs.io/en/stable/ .

[9] Konstantinov, Andrei V., and Lev V. Utkin. “A Generalized Stacking for Implementing Ensembles of Gradient Boosting Machines.” In Studies in Systems, Decision and Control, 3–16, 2021. https://link.springer.com/chapter/10.1007/978-3-030-67892-0_1.

[10] Documentation of Scikit-Learn “Scikit-Learn” 2007 https://scikit-learn.org/0.21/documentation.html

[11] Lecun, Yann, et al. "Gradient-Based Learning Applied to Document Recognition." Proceedings of the IEEE 86, no. 11 (2007): 2278-2324. doi: 10.1109/PROC.2007.898639

[12] Zhang, Zhongheng, Yiming Zhao, Aran Canes, Dan Steinberg, and Olga Lyashevska. 2019. “Predictive Analytics with Gradient Boosting in Clinical Medicine.” Annals of Translational Medicine 7 (7): 152–52. https://atm.amegroups.org/article/view/24543/23475.

[13] Al Shourbaji, Ibrahim, Na Helian, Yi Sun, Abdelazim G. Hussien, Laith Abualigah, and Bushra Elnaim. 2023. “An Efficient Churn Prediction Model Using Gradient Boosting Machine and Metaheuristic Optimization.” Scientific Reports 13 (1): 14441. https://www.nature.com/articles/s41598-023-41093-6.

[14] Manley, William, Tam Tran, Melissa Prusinski, and Dustin Brisson. “Modeling Tick Populations: An Ecological Test Case for Gradient Boosted Trees.” bioRxiv : the preprint server for biology, November 29, 2023. https://pmc.ncbi.nlm.nih.gov/articles/PMC10054924/#:~:text=The%20rapidly%20expanding%20environmental%20data,development%20of%20public%20health%20strategies.